Birzeit University

Faculty of Engineering and Technology

Department of Computer Science

Database Management Systems (COMP333)

Phase 4

Electronics Store Information Management System

Prepared by:

Jamil Habash 1220194

Abdallah Kokash 1220116

Instructor: Bassem Sayrafi

Section: 2

Date: 10/6/2025

Table Of Contents

Table Of Figures 2
Introduction: 3
Client information: 3
Technology: 4
Project details: 5
- **Entities:** 5
- **Relationships:** 6
- **Queries:** 7
- **Entity-Relationship diagram:** 8
- **Normalization of Tables** 9
  - **1- Customer Table** 9
  - **2- Employee Table** 9
  - **3- Order Table** 9
  - **4- Order_Details Table** 9
  - **5- Product Table** 10
  - **6- Model Table** 10
  - **7- Company Table** 10
  - **8- Inventory_Record Table** 10
  - **9- Payment Table** 10
  - **10- Purchase Table** 11
  - **11- Purchase_Detail Table** 11

Conclusion 12

Table Of Figures

Figure 1: ERD diagram 8

Introduction:

The client wants to implement a database system for their electronics store, which sells a variety of products such as laptops, printers, mice, keyboards, and other accessories. The system will help manage inventory, track sold products, and organize supplier (agent) information efficiently.

This system will help the client deal with unorganized data and help keep track of supplier details, inventory, trends and more. The database will resolve these issues by keeping a clear record of all incoming and outgoing products, ensuring accurate stock management. This system will also provide reports to help the client make informed business decisions, reduce losses, and improve overall store efficiency.

Client information:

The client is the manager of a local electronics store called **الرحال**, which operates under the business number **(No. 2420392)**. With a focus on providing high-quality electronic products such as laptops, printers, mice, and keyboards, the client oversees all aspects of store management, including inventory control, supplier relations, and sales tracking.

Technology:

Name	Processor	RAM	Graphic card	Storage	OS
Jamil Habash	Intel core I5 12400F	16 GB DDR4	GeForce RTX 3050	512GB SSD	Windows 10
Abdallah Kokash	Intel core I7 11370H	16 GB DDR4	GeForce RTX 3060	1T SSD	Windows 10

In this project, we are utilizing a combination of technologies to develop an efficient database-driven application. Python, along with Flask, serves as the backend framework, providing a lightweight yet powerful environment for handling server-side logic and database interactions. We are using MySQL as the database management system to store, manage, and retrieve data efficiently. PyCharm is our primary development environment, offering advanced coding and debugging features. For the frontend, we are leveraging HTML and CSS to create a responsive and user-friendly interface.

Project details:

Entities:

- **Products** have an ID, Price, Product Type, Warranty, Picture, and Descriptions.
- **Employees** have an ID, Name, Phone Number, Address, Email, Birthdate, Salary and Pass.
- **Customers** have an ID, Phone Number, Address, Name, Email and Pass.
- **Orders** have an ID, Customer ID, Employee ID, Total Price, and Order date.
- **Order Details** contain an Order ID, Product ID, discount and quantity.
- **Companies** have an ID, Name, Address, and Mail.
- **Models** have an ID, Name and Company ID, and manufactural date.
- **Purchase** has an ID, Employee ID, Purchase date and Total cost.
- **Purchase Detail** has ID, Model ID, Quantity, and Cost per unit.
- **Inventory Record** has Model ID and Quantity.
- **Payment** has Order ID, Payment method, Payment identifier and Payment key.

Relationships:

- A **Customer** can place multiple **Orders**, but each **Order** belongs to only one **Customer**.
- An **Employee** can handle multiple **Orders**, but each **Order** is processed by only one **Employee**.
- Each **Product** belongs to a specific **Model**, but a **Model** can have multiple **Products**.
- Each **Model** is created by one **Company**, but a **Company** can manufacture multiple **Models**.
- An **Order** can have multiple **Details** and a **Detail** is tight to an **Order**.
- A **product** has one **Detail** and a **Detail** belongs to one **Product**.
- **A Purchase** contains multiple **Purchase Details**, and each **Purchase Detail** is tied to one **Purchase**.
- Each **Purchase Detail** refers to exactly one **Model**, but a **Model** can appear in multiple **Purchase Details**.
- Each **Purchase** is initiated by one **Employee**, but an **Employee** can initiate multiple **Purchases**.
- An **Inventory Record** tracks the quantity of a specific **Model**, and a **Model** can have one **Inventory Record.**
- An **Order** has a **Payment** method and a **Payment** method belong to only one **Order.**

Queries:

- Retrieve all customer details.
- Retrieve the orders placed by each customer.
- Retrieve the Purchase orders placed by each employee.
- Retrieve the details of all orders handled by a specific employee.
- Retrieve the total orders handled by each employee.
- Retrieve all products and their prices.
- Retrieve the details of all orders that contain a specific product.
- Retrieve the total revenue generated from each product.
- Retrieve details of orders placed within a specific date range.
- Retrieve the most frequently ordered product.
- Retrieve the orders processed by each employee.
- Retrieve the top 5 customers by spending.
- Retrieve all orders in a specific price range.
- Retrieve the latest products model added to the database.
- Retrieve the details of all orders placed by a specific customer.
- Retrieve the details of all orders Handled by a specific employee.
- Retrieve all orders in a specific time range.
- Retrieve the total profit generated by the store.
- Retrieve the total expenses generated by the store.
- Retrieve the total revenue generated by the store.
- Retrieve all products that are low in stock.
- Retrieve the total amount of customers and employees in the store.
- Retrieve all Purchase orders in a specific time range.

Entity-Relationship diagram:

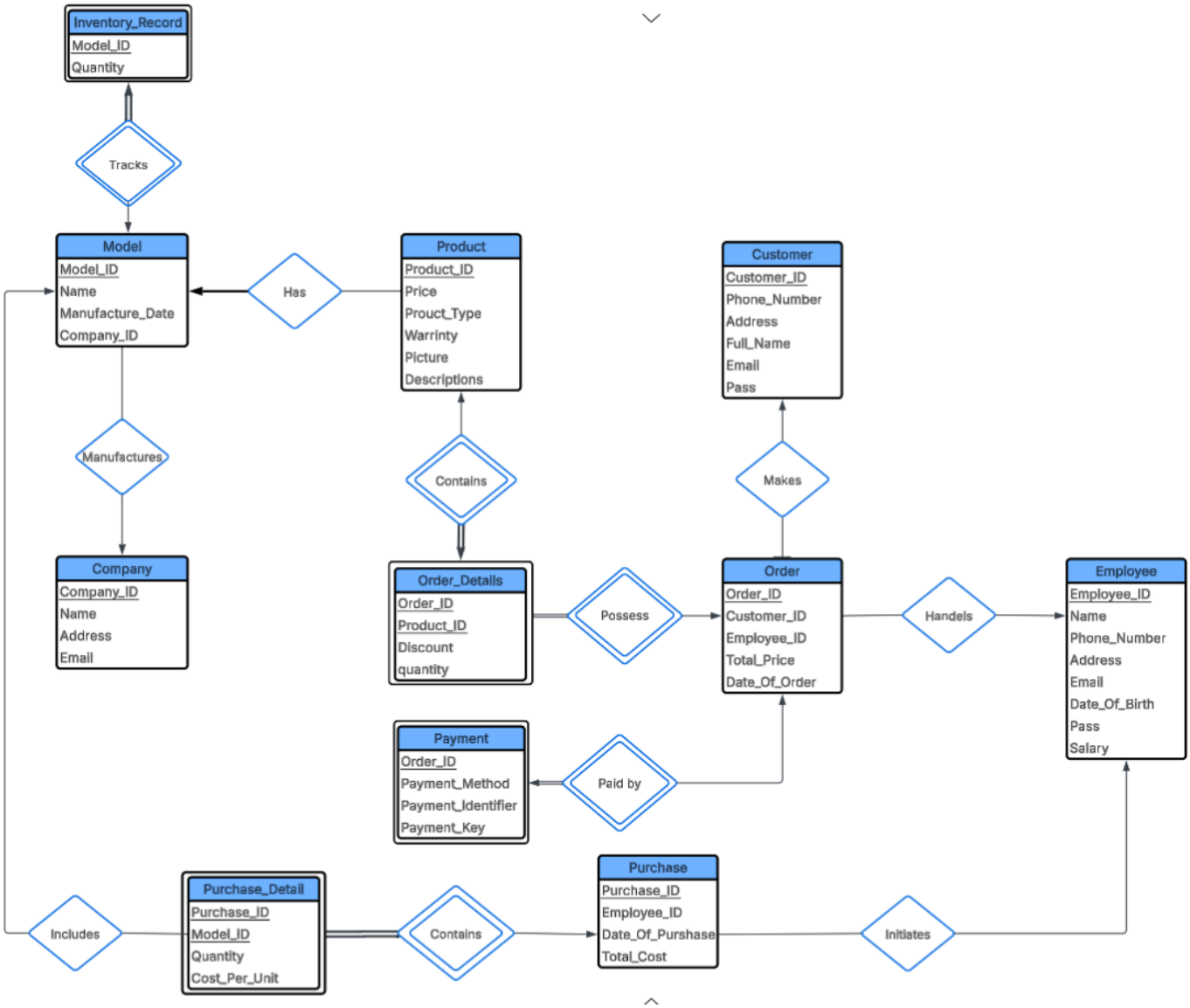

Figure 1: ERD diagram

Normalization of Tables

1- Customer Table

Customer (Customer_ID, Phone_Number, Address, Full_Name, Email, Pass)

- **1NF**: All attributes have atomic values.
- **2NF**: Customer_ID is the primary key, and all other fields depend on it completely.
- **3NF**: No non-key field depends on another non-key field. For example, Email doesn't depend on Full_Name. So, this table is in 3NF.

2- Employee Table

Employee (Employee_ID, Email, Phone_Number, Address, Pass, Emp_Name ,Date_Of_Birth, Pass, Salary)

- **1NF**: All attributes have atomic values.
- **2NF**: Employee_ID is the primary key, and all fields depend fully on it.
- **3NF**: No non-key field depends on another non-key field. So, the table is in 3NF.

3- Order Table

Order (Order_ID, Customer_ID, Employee_ID, Total_Price, Date_Of_Order)

- **1NF**: All attributes have atomic values.
- **2NF**: Order_ID is the primary key, and all other fields depend on it.
- **3NF**: No non-key field depends on another non-key field. So, this table is also in 3NF.

4- Order_Details Table

Order_Details (Order_ID, Product_ID, Discount, quantity)

- **1NF**: All attributes have atomic values.
- **2NF**: Primary key is a combination of Order_ID and Product_ID. Discount and Quantity depend on both.
- **3NF**: No non-key field depends on another non-key field. The table is in 3NF.

5- Product Table

Product (Product_ID, Price, Product_Type, Warranty, Picture, Descriptions)

- **1NF**: All attributes have atomic values.
- **2NF**: Product_ID is the primary key, and all fields depend on it.
- **3NF**: No non-key field depends on another non-key field. So, it's in 3NF.

6- Model Table

Model (Model_ID, Model_Name, Manufacture_Date, Company_ID)

- **1NF**: All attributes have atomic values.
- **2NF**: Model_ID is the primary key; all other fields depend fully on it.
- **3NF**: No non-key field depends on another non-key field. For example, Name does not depend on Company_ID. So, it's in 3NF.

7- Company Table

Company (Company_ID, Company_Name, Address, mail)

- **1NF**: All attributes have atomic values.
- **2NF**: Company_ID is the key, and all fields depend on it.
- **3NF**: No non-key field depends on another non-key field. So, it's in 3NF.

8- Inventory_Record Table

Inventory_Record (Model_ID, Quantity)

- **1NF**: All attributes have atomic values.
- **2NF**: Model_ID is the key; all fields depend on it.
- **3NF**: No non-key field depends on another non-key field. The table is normalized.

9- Payment Table

Payment (Order_ID, Payment_Method, Payment_Identifier, Payment_Key)

- **1NF**: All attributes have atomic values.
- **2NF**: Order_ID is the key; other fields depend on it.
- **3NF**: No non-key field depends on another non-key field. So it is in 3NF.

10- Purchase Table

Purchase (Purchase_ID, Employee_ID, Date_Of_Purchase, Total_Cost)

- **1NF**: All attributes have atomic values.
- **2NF**: Purchase_ID is the key; all fields are fully dependent on it.
- **3NF**: No non-key field depends on another non-key field. This table is also in 3NF.

11- Purchase_Detail Table

Purchase_Detail (Purchase_ID, Model_ID, Quantity, Cost_Per_Unit)

- **1NF**: All attributes have atomic values.
- **2NF**: The primary key is (Purchase_ID, Model_ID); Quantity and Cost_Per_Unit depend on both.
- **3NF**: No non-key field depends on another non-key field. So, this table is in 3NF.

Conclusion

In conclusion, the Electronics Store Information Management System successfully fulfills the client's requirements by providing a structured and efficient way to manage store operations. Through careful design of the database schema and normalization of all tables up to the third normal form (3NF), the system ensures data consistency, integrity, and elimination of redundancy. The use of technologies such as Python with Flask, MySQL, and a responsive HTML/CSS interface has enabled the development of a functional and user-friendly platform. This system not only supports essential operations like order processing, inventory tracking, and supplier management but also offers valuable queries for reporting and analysis.